

1. ZADATAK

PRII – G1

Izvršiti definiciju funkcija na način koji odgovara opisu (komentarima) datom neposredno uz pozive ili nazive funkcija. Možete dati komentar na bilo koju liniju code-a koju smatrate da bi trebalo unaprijediti ili da će eventualno uzrokovati grešku prilikom kompajliranja. Također, možete dodati dodatne funkcije ili metode koje će vam olakšati implementaciju programa.

```
#include <iostream>
using namespace std;

string crt = "\n-----\n";

string PORUKA_TELEFON = crt +
"TELEFONE ISKLJUCITE I ODLOZITE U TORBU, DZEP ILI DRUGU LOKACIJU VAN
DOHVATA.\n"
"CESTO SE NA TELEFONIMA (PRO)NALAZE PROGRAMSKI KODOVI KOJI MOGU BITI
ISKORISTENI ZA\n"
"RJESAVANJE ISPITNOG ZADATKA, STO CE, U SLUCAJU PRONALASKA, BITI
SANKCIONISANO.\n\n"
"SVI KOD KOJIH BUDE PRONADJEN TELEFON ILI NEKI DRUGI UREDJAJ KOJI MOZE
SADRZAVATI\n"
"MATERIJAL ILI RJESENJA ISPITNIH ZADATAKA CE BITI UDALJENI SA ISPITA I
BIT CE POKRENUT\n"
"POSTUPAK PROTIV NJIH" + crt;

string PORUKA_ISPIT = crt +
"0. PROVJERITE DA LI ZADACI PRIPADAJU VASOJ GRUPI (G1/G2)\n"
"1. SVE KLASSE SA DINAMICKOM ALOKACIJOM MORAJU IMATI ISPRAVAN
DESTRUKTOR\n"
"2. IZOSTAVLJANJE DESTRUKTORA ILI NJEGOVIH DIJELOVA BIT CE OZNACENO
KAO TM\n"
"3. ATRIBUTI, METODE I PARAMETRI MORAJU BITI IDENTICNI ONIMA U TESTNOJ
MAIN FUNKCIJI,\n"
"    OSIM AKO POSTOJI JASNO OPISAN RAZLOG ZA MODIFIKACIJU\n"
"4. IZUZETKE BACAJTE SAMO TAMO GDJE JE IZRICITO NAGLASENO\n"
"5. SVE METODE KOJE SE POZIVAJU U MAIN-U MORAJU POSTOJATI.\n"
"    AKO NEMATE ZELJENU IMPLEMENTACIJU, OSTAVITE PRAZNO TIJELO ILI
VRATITE DEFAULT VRIJEDNOST\n"
"6. U MAIN FUNKCIJI MOZETE DODAVATI TESTNE PODATKE I POZIVE PO
VLASTITOM IZBORU\n"
"7. TESTIRAJTE PROGRAM U OBA MODA (F5 i Ctrl+F5)" + crt;

char* AlocirajTekst(const char* tekst) {
    if (tekst == nullptr)        return nullptr;
    size_t velicina = strlen(tekst) + 1;
    char* noviTekst = new char[velicina];
    strcpy_s(noviTekst, velicina, tekst);
    return noviTekst;
}

enum VrstaDostave { STANDARDNA, BRZA, MEDJUNARODNA };
const char* VrstaDostaveOznake[] = { "ST", "BR", "MD" };
const char* VrstaDostaveNazivi[] = { "STANDARDNA", "BRZA",
"MEDJUNARODNA" };

enum StatusPosiljke { KREIRANA, PREUZETA, U_TRANZITU, ISPORUCENA,
OTKAZANA };
```

```

const char* StatusPosiljkeNazivi[] = { "KREIRANA", "PREUZETA", "U
TRANZITU", "ISPORUCENA", "OTKAZANA" };

template<class T1, class T2, int max>
class Kolekcija {
    T1* _elementi1[max];
    T2* _elementi2[max];
    int _trenutno;
public:
    Kolekcija() : _trenutno(0) {
        for (int i = 0; i < max; i++) {
            _elementi1[i] = nullptr;
            _elementi2[i] = nullptr;
        }
    }
    int GetTrenutno() const { return _trenutno; }
    T1& GetElement1(int indeks) { return *_elementi1[indeks]; }
    T2& GetElement2(int indeks) { return *_elementi2[indeks]; }
    const T1& GetElement1(int indeks) const { return
*_elementi1[indeks]; }
    const T2& GetElement2(int indeks) const { return
*_elementi2[indeks]; }
    T1& operator[](int indeks) { return *_elementi1[indeks]; }
    friend ostream& operator<<(ostream& COUT, const Kolekcija& obj) {
        for (int i = 0; i < obj.GetTrenutno(); i++)
            COUT << obj.GetElement1(i) << " " << obj.GetElement2(i) <<
endl;
        return COUT;
    }
    ~Kolekcija() {
        for (int i = 0; i < _trenutno; i++) {
            delete _elementi1[i];
            delete _elementi2[i];
            _elementi1[i] = nullptr;
            _elementi2[i] = nullptr;
        }
        _trenutno = 0;
    }
};

class DatumVrijeme {
    int* _dan, * _mjesec, * _godina, * _sati, * _minute;
public:
    DatumVrijeme(int dan = 1, int mjesec = 1, int godina = 2000,
        int sati = 0, int minute = 0) {
        _dan = new int(dan);
        _mjesec = new int(mjesec);
        _godina = new int(godina);
        _sati = new int(sati);
        _minute = new int(minute);
    }
    friend ostream& operator<<(ostream& COUT, const DatumVrijeme& obj)
{
        //ToString vraca datum i vrijeme u formatu DD.MM.GGGG HH:MM,
        COUT << obj.ToString();
        return COUT;
    }
    ~DatumVrijeme() {

```

```

        delete _dan;
        delete _mjesec;
        delete _godina;
        delete _sati;
        delete _minute;
        _dan = _mjesec = _godina = _sati = _minute = nullptr;
    }
};

class Posiljka {
    char* _oznaka;
    char* _sadrzaj;
    char* _imePrezimePrimaoca;
    char* _emailPrimaoca;
    VrstaDostave _vrstaDostave;
    int _redniBroj;
    Kolekcija<StatusPosiljke, DatumVrijeme, 10> _statusi;
    void Dealociraj() {
        delete[] _oznaka;
        delete[] _sadrzaj;
        delete[] _imePrezimePrimaoca;
        delete[] _emailPrimaoca;
        _oznaka = _sadrzaj = _imePrezimePrimaoca = _emailPrimaoca =
nullptr;
    }
public:
    const char* GetOznaka() const { return _oznaka; }
    const char* GetSadrzaj() const { return _sadrzaj; }
    const char* GetImePrezimePrimaoca() const { return
_imePrezimePrimaoca; }
    const char* GetEmailPrimaoca() const { return _emailPrimaoca; }
    VrstaDostave GetVrstaDostave() const { return _vrstaDostave; }
    int GetRedniBroj() const { return _redniBroj; }
    Kolekcija<StatusPosiljke, DatumVrijeme, 10>& GetStatusi() { return
_statusi; }
    StatusPosiljke GetTrenutniStatus()const {
        return _statusi.GetElement1(_statusi.GetTrenutno() - 1);
    }
    friend ostream& operator<<(ostream& COUT, const Posiljka& obj) {
        // ToString vraca podatke u formatu:
        // oznaka | ime i prezime | vrsta dostave | sadrzaj | trenutni
status
        // BH-BR-0042-AB | Amina Buric | BRZA | Dokumenti | KREIRANA
        COUT << obj.ToString();
        return COUT;
    }
    ~Posiljka() {
        Dealociraj();
    }
};

class DostavnaSluzba {
    char* _naziv;
    vector<Posiljka> _posiljke;
public:
    DostavnaSluzba(const char* naziv = "") {
        _naziv = AlocirajTekst(naziv);
    }
};

```

```

const char* GetNaziv() const { return _naziv; }
vector<Posiljka>& GetPosiljke() { return _posiljke; }
~DostavnaSluzba() {
    delete[] _naziv;
    _naziv = nullptr;
}
};

const char* GetOdgovorNaPrvoPitanje() {
    cout << "Pitanje -> Pojasnite da li se svako nasljedjivanje moze
posmatrati kao polimorfizam i obrnuto? Navedite konkretne
primjere.\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

const char* GetOdgovorNaDrugoPitanje() {
    cout << "Pitanje -> Pojasnite razloge kojima bi se implementator
klase mogao voditi pri donosenju odluke da sve metode u klasi
(ne)proglasi virtualnim?\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

int main() {
    cout << PORUKA_TELEFON; cin.get();
    cout << PORUKA_TELEFON; cin.get();
    cout << PORUKA_ISPIT;
    cin.get();
    system("cls");

    cout << GetOdgovorNaPrvoPitanje() << crt;
    cin.get();
    cout << GetOdgovorNaDrugoPitanje() << crt;
    cin.get();

    /*
    Funkcija GenerisiOznaku generise oznaku posiljke u formatu: BH-VD-
BBBB-IN
    Potpis funkcije treba biti:
    string GenerisiOznaku(const char* imePrezime, int redniBroj,
VrstaDostave vrstaDostave)

    BH    -> fiksna oznaka drzave,
    VD    -> vrsta dostave: ST (standardna), BR (brza), MD
(medjunarodna),
    BBBB -> redni broj posiljke popunjen nulama na slobodnim mjestima,
    IN    -> inicijali imena i prezimena primaoca.

    Kod imena koje sadrzi vise rijeci koriste se inicijal prve i
posljednje
    rijeci. Redni broj mora biti u rasponu 1-9999.
    Za neispravne podatke funkcija vraca "BH-XX-0000-XX".
    */
    if (GenerisiOznaku("Amina Buric", 42, BRZA) == "BH-BR-0042-AB")
        cout << "Oznaka OK" << crt;
    if (GenerisiOznaku("Goran Skondric", 7, STANDARDNA) == "BH-ST-
0007-GS")
        cout << "Oznaka OK" << crt;

```

```

if (GenerisiOznaku("Ana Marija Kovac", 156, MEDJUNARODNA) ==
    "BH-MD-0156-AK")
    cout << "Oznaka OK" << crt;
if (GenerisiOznaku("Amina", 42, BRZA) == "BH-XX-0000-XX" &&
    GenerisiOznaku("Amina Buric", 0, BRZA) == "BH-XX-0000-XX" &&
    GenerisiOznaku("Amina Buric", 10000, BRZA) == "BH-XX-0000-XX"
&&
    GenerisiOznaku("Amina Buric", 42, (VrstaDostave)99) == "BH-XX-
0000-XX")
    cout << "Neispravni podaci za oznaku OK" << crt;

/*
Koristeci regex, funkcija ValidirajOznaku provjerava da li je
oznaka
zapisana u prethodno definisanom formatu. Dozvoljene su iskljucivo
oznake
vrsta ST, BR i MD, redni broj mora imati cetiri cifre i ne moze
biti 0000,
a inicijali moraju biti zapisani velikim slovima.
Potpis funkcije treba biti:
bool ValidirajOznaku(const string& oznaka)
*/
if (ValidirajOznaku("BH-BR-0042-AB"))
    cout << "OZNAKA VALIDNA" << crt;
if (!ValidirajOznaku("BH-EX-0042-AB") &&
    !ValidirajOznaku("BH-BR-042-AB") &&
    !ValidirajOznaku("BH-BR-0042-Ab") &&
    !ValidirajOznaku("BH-BR-0000-AB") &&
    !ValidirajOznaku("bh-BR-0042-AB"))
    cout << "OZNAKA NIJE VALIDNA" << crt;
if (ValidirajOznaku(GenerisiOznaku("Maid Ramic", 284, BRZA)))
    cout << "GENERISANA OZNAKA VALIDNA" << crt;

Kolekcija<int, string, 5> brojevi;
brojevi.Dodaj(10, "Deset");
brojevi.Dodaj(20, "Dvadeset");
brojevi.Dodaj(30, "Trideset");
cout << brojevi << crt;

/*
DodajNaPoziciju dodaje novi par na lokaciju/indeks definisanu
prvim parametrom,
pomjera postojece pokazivace udesno i vraca trenutno stanje
kolekcije (izmijenjeni tj. prosireni objekat).
U slucaju popunjene kolekcije ili neispravne lokacije potrebno je
baciti izuzetak.
*/
Kolekcija<int, string, 5> prosireniBrojevi =
    brojevi.DodajNaPoziciju(1, 15, "Petnaest");
cout << prosireniBrojevi << crt;

/*
UkloniSaPozicije uklanja par sa prosljedjene pozicije/indeksa,
preostale elemente
pomjera ulijevo i vraca uklonjeni par u obliku pair<T1, T2>.
Za poziciju izvan opsega baca se izuzetak.
*/

```

```

    pair<int, string> uklonjeni =
    prosireniBrojevi.UkloniSaPozicije(2);
    cout << "Uklonjeno: " << uklonjeni.first << " " <<
    uklonjeni.second << crt;
    cout << "Preostali elementi:" << crt << prosireniBrojevi;

    try {
        prosireniBrojevi.UkloniSaPozicije(10);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }

    try {
        Kolekcija<int, string, 3> punaKolekcija;
        punaKolekcija.Dodaj(1, "Jedan");
        punaKolekcija.Dodaj(2, "Dva");
        punaKolekcija.Dodaj(3, "Tri");
        punaKolekcija.DodajNaPoziciju(1, 4, "Cetiri");
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }

    Kolekcija<int, string, 5> kopijaBrojeva = brojevi;
    kopijaBrojeva[0] = 100;
    Kolekcija<int, string, 5> dodijeljeniBrojevi;
    dodijeljeniBrojevi = brojevi;
    dodijeljeniBrojevi[1] = 200;
    cout << "Original:" << crt << brojevi;
    cout << "Kopija:" << crt << kopijaBrojeva;
    cout << "Dodijeljeni objekat:" << crt << dodijeljeniBrojevi;

    DatumVrijeme kreirana(1, 7, 2026, 8, 0);
    DatumVrijeme preuzeta(1, 7, 2026, 9, 0);
    DatumVrijeme tranzit(1, 7, 2026, 10, 0);
    DatumVrijeme isporucena(1, 7, 2026, 11, 0);

    /*
    ToString vraca datum i vrijeme u formatu DD.MM.GGGG HH:MM,
    ukljucujuci
    pocetne nule.
    */
    cout << kreirana.ToString() << crt; //01.07.2026 08:00
    if (preuzeta > kreirana)
        cout << "Vrijeme preuzimanja je nakon vremena kreiranja" <<
crt;
    DatumVrijeme kopijaPreuzeta(preuzeta);
    if (kopijaPreuzeta == preuzeta && !(kreirana == preuzeta))
        cout << "Provjera vremena, OK." << crt;
    /*
    Konstruktor Posiljke generise oznaku i evidentira pocetni status
    KREIRANA
    sa prosljedjenim vremenom kreiranja.
    */
    Posiljka dokumenti("Dokumenti", "Amina Buric", "amina@fit.ba",
        BRZA, 42, kreirana);

```

```
Posiljka knjige("Knjige", "Goran Skondric", "goran@fit.ba",
    STANDARDNA, 7, kreirana);
Posiljka oprema("Oprema", "Ana Marija Kovac", "ana@fit.ba",
    MEDJUNARODNA, 156, kreirana);

/*
ToString vraća podatke u formatu:
oznaka | ime i prezime | vrsta dostave | sadržaj | trenutni status
BH-BR-0042-AB | Amina Buric | BRZA | Dokumenti | KREIRANA
*/
cout << dokumenti.ToString() << crt;
if (dokumenti.ToString() ==
    "BH-BR-0042-AB | Amina Buric | BRZA | Dokumenti | KREIRANA")
    cout << "Posiljka ToString OK" << crt;

/*
DodajStatus dodaje status samo ako je vrijeme veće od vremena
posljednjeg statusa i ako je promjena statusa dozvoljena.

Dozvoljen slijed statusa je: KREIRANA -> PREUZETA -> U_TRANZITU ->
ISPORUCENA

OTKAZANA se može evidentirati iz bilo kojeg statusa koji nije
završni.
Nakon statusa ISPORUCENA ili OTKAZANA nisu dozvoljene nove
promjene.
Preskakanje, ponavljanje statusa i hronoloski neispravno vrijeme
vracaju
false, bez izmjene kolekcije statusa.
*/
if (dokumenti.DodajStatus(PREUZETA, preuzeta))
    cout << "Status PREUZETA dodan" << crt;
if (dokumenti.DodajStatus(U_TRANZITU, tranzit))
    cout << "Status U TRANZITU dodan" << crt;
if (dokumenti.DodajStatus(ISPORUCENA, isporucena))
    cout << "Status ISPORUCENA dodan" << crt;
if (!dokumenti.DodajStatus(OTKAZANA, isporucena))
    cout << "Promjena završnog statusa nije dozvoljena" << crt;

if (!knjige.DodajStatus(U_TRANZITU, preuzeta))
    cout << "Preskakanje statusa nije dozvoljeno" << crt;
if (oprema.DodajStatus(OTKAZANA, preuzeta))
    cout << "Posiljka otkazana" << crt;

Posiljka kopijaDokumenata = dokumenti;
cout << kopijaDokumenata << crt;

DostavnaSluzba brzaPosta("Brza posta");

/*
DodajPosiljku dodaje posiljku u dostavnu službu. Nije dozvoljeno
dodati
dvije posiljke sa istim rednim brojem ili istom oznakom. U slučaju
duplikata metoda baca izuzetak.
*/
brzaPosta.DodajPosiljku(knjige);
brzaPosta.DodajPosiljku(oprema);
try {
```

```

        brzaPosta.DodajPosiljku(knjige);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }

    /*
    PronadjiPosiljku vraca pokazivac na posiljku sa prosljedjenom
    oznakom.
    Ako posiljka nije pronadjena, metoda vraca nullptr.
    */
    string oznakaKnjige = knjige.GetOznaka();
    Posiljka* pronadjena =
        brzaPosta.PronadjiPosiljku(oznakaKnjige);
    if (pronadjena != nullptr)
        cout << "Pronadjena posiljka: " << pronadjena->GetOznaka() <<
crt;
    if (brzaPosta.PronadjiPosiljku("BH-ST-9999-XX") == nullptr)
        cout << "Nepostojeca posiljka nije pronadjena" << crt;

    /*
    EvidentirajStatus pronalazi posiljku na osnovu oznake i pokusava
    dodati
    novi status koristeći pravila metode DodajStatus.

    Nakon svakog uspješno evidentiranog statusa potrebno je u zasebnom
    threadu
    poslati obavijest primaocu. Obavijest se ne salje ako posiljka ne
    postoji ili status nije dodan, te u tim slučajevima metoda vraca
    false.

    Primjer sadržaja obavijesti:
    -----
    To: goran@fit.ba
    From: info@brzaposta.ba
    Subject: Promjena statusa posiljke

    Postovani Goran Skondric,

    Posiljka BH-ST-0007-GS je promijenila status u PREUZETA.
    Vrijeme evidentiranja: 01.07.2026 09:00

    Hvala sto koristite nase usluge.
    BrzaPosta Tim
    -----
    -----
    */
    if (brzaPosta.EvidentirajStatus(
        oznakaKnjige, PREUZETA, preuzeta))
        cout << "Status evidentiran i obavijest poslana" << crt;
    if (!brzaPosta.EvidentirajStatus(oznakaKnjige, ISPORUCENA,
isporucena))
        cout << "Obavijest nije poslana za neispravnu promjenu
statusa" << crt;
    if (!brzaPosta.EvidentirajStatus("NEPOSTOJECA", PREUZETA,
preuzeta))

```



```
        cout << "Obavijest nije poslana za nepostojecu posiljku" <<
crt;

    /*
    PosiljkePoStatusu vraca vector pokazivaca na sve posiljke ciji
    trenutni
    status odgovara proslijedjenom statusu.
    */
    vector<Posiljka*> preuzetePosiljke =
        brzaPosta.PosiljkePoStatusu(PREUZETA);

    for (auto posiljka : preuzetePosiljke)
        cout << posiljka->GetOznaka() << " -> "
        << StatusPosiljkeNazivi[(int)posiljka->GetTrenutniStatus()] <<
crt;

    /*
    StatistikaPoStatusu vraca kolekciju koja za svaki status,
    redoslijedom
    definisanim enumom StatusPosiljke, sadrzi status i broj posiljki
    koje se
    trenutno nalaze u tom statusu. U rezultat se dodaje svih pet
    statusa,
    ukljucujuci i one za koje je broj posiljki jednak nuli.
    */
    Kolekcija<StatusPosiljke, int, 5> statistika =
        brzaPosta.StatistikaPoStatusu();
    for (int i = 0; i < statistika.GetTrenutno(); i++)
        cout << StatusPosiljkeNazivi[(int)statistika.GetElement1(i)]
        << " -> " << statistika.GetElement2(i) << crt;

    DostavnaSluzba kopijaSluzbe = brzaPosta;
    cout << kopijaSluzbe.GetNaziv() << " ima "
        << kopijaSluzbe.GetPosiljke().size() << " posiljki" << crt;

    cin.get();
    return 0;
}
```